

FloorplanFit

Authoritative DXF adaptation and synchronized plan sets

SYSTEM ARCHITECTURE AND OPERATIONS MANUAL

61 DAYS

INVESTED

STACK: C#, .NET 10, Avalonia, CommunityToolkit.Mvvm, Microsoft.Extensions.Hosting, Microsoft.Data.Sqlite, SQLite, IxMilia.Dxf, HttpClient, OpenAI Responses API, xUnit, coverlet

Contents

01	1. System charter
02	2. System operating model
03	3. Loop 1 - Curate
04	4. Loop 2 - Adapt
05	5. HousePlanSet synchronization and export
06	6. Layered architecture
07	7. Data structures and relationships
08	8. SQLite persistence lifecycle
09	9. Provenance, artifacts, and auditability
10	10. Desktop and operator workflows
11	11. Technology stack and configuration
12	12. Capability matrix
13	13. Failure modes and safeguards
14	14. Extension strategy
15	15. Strategic relationship to Point.ai
16	16. Operating principles

1. System charter

FloorplanFit is a local-first desktop system for converting architect-authored DXF drawings into curated, reusable house-plan assets, adapting those assets to site constraints, and synchronizing the accepted adjustment across a related set of technical sheets.

The system is not a general CAD editor and does not replace AutoCAD. It controls a narrower workflow:

- import and preserve the original CAD source;
- extract technical evidence without treating extraction as unquestionable truth;
- let an operator curate and publish a canonical FloorPlan;
- analyze whether that canonical plan fits a SitePlan;
- apply deterministic geometry changes only in authorized adjustment zones;
- project the same accepted adjustment to dependent sheets;
- export traceable DXF outputs with manifests, audits, and fail-closed quality gates.

1.1 Product authority model

FloorplanFit uses an explicit hierarchy of authority:

- **Managed source DXF** is the immutable technical source of truth.
- **Human-reviewed published curation** is the canonical semantic interpretation of that source.
- **The canonical FloorPlan** owns site fit and geometry-adjustment decisions.
- **The canonical adjustment recipe** owns propagation of the accepted change.
- **Dependent-sheet registrations and capability policies** determine how that recipe may be projected.
- **Typed verification and audit artifacts** determine whether an export may be treated as automatically ready.
- **AI output is advisory only.** It may rank or explain valid options but cannot invent geometry authority, compression capacity, registration truth, or export acceptance.

This hierarchy prevents a convenient but dangerous failure mode: allowing an inferred value, preview approximation, or language-model response to silently replace the technical source.

1.2 Scope boundary

FloorplanFit owns:

- managed DXF import;
- source hashing, unit context, and version identity;
- CAD artifact extraction;
- operator review and curation;
- publication of canonical FloorPlan versions;
- measurement relationships and authorized adjustment zones;
- SitePlan intake and fit analysis;
- deterministic adjustment proposals;
- source-preserving FloorPlan and SitePlan export;
- HousePlanSet sheet registration and projection;
- package manifests, verification, quality signals, and audit records.

FloorplanFit does not currently own:

- arbitrary CAD drafting;
- legal interpretation of zoning or setback rules;
- universal parsing of every DXF convention;
- automatic semantic compression of RoofPlan or Facade/Elevation sheets;
- bidirectional synchronization from dependent sheets back into the FloorPlan;
- autonomous approval of inferred geometry;
- a model-training pipeline.

2. System operating model

The product is organized into three connected workflows.

```
Loop 1: Curate
Raw FloorPlan DXF
-> managed source
-> extraction
-> human review
-> Fit policy
-> published canonical FloorPlan

Loop 2: Adapt
Published FloorPlan + SitePlan
-> buildable-area analysis
-> deterministic fit options
-> accepted placement and compression
-> adjusted FloorPlan and SitePlan output

HousePlanSet
Canonical adjustment
-> dependent-sheet registration
-> capability-aware projection
-> confirmation
-> multi-sheet package
-> manifest and audit
```

The loops deliberately separate reusable knowledge from site-specific decisions:

- Loop 1 is performed once per house-plan version and produces reusable canonical truth.
- Loop 2 is performed for each site and consumes the published result.
- HousePlanSet reuses the accepted Loop 2 decision across the complete technical drawing set.

3. Loop 1 - Curate

Loop 1 turns an external FloorPlan DXF into a governed technical asset.

3.1 Intake

The import workflow:

- reads source metadata and DXF version;
- resolves the drawing unit;
- creates a measurement context;
- calculates a SHA-256 hash;
- copies the file into managed storage;
- creates or resolves a FloorPlanTemplate;
- creates an incremental FloorPlanVersion;

- preserves the original file identity and managed source path.

The original drawing remains the source. Extraction and curation are stored as layers of interpretation rather than destructive replacements.

3.2 Extraction

Extraction proposes separate CAD artifact families:

- wall candidates;
- room labels;
- opening candidates;
- opening labels;
- fixed plan components and fixtures;
- protected detail assemblies;
- native dimensions;
- shared geometry paths and segments.

Each artifact retains source-facing information such as entity reference, layer, geometry, confidence, detection notes, and order. The extraction profile uses known Pointe Homes conventions and should not be represented as a universal DXF understanding engine.

3.3 Review

The Desktop review workbench presents a CAD-like view with:

- zoom and pan;
- selection and hit testing;
- layer-aware and semantic rendering;
- artifact exclusion and restoration;
- classification correction;
- position correction;
- label-size correction;
- native-dimension inspection and editing;
- manual wall creation;
- Fit measurement and adjustment controls.

The operator does not redraw the full plan. Extraction supplies candidate evidence; review corrects the subset that matters.

3.4 Curation

A FloorPlanCuration belongs to one FloorPlanVersion and has a controlled lifecycle:

```
Draft
-> editable decisions and Fit data
-> readiness validation
-> Published
-> active canonical curation
-> later edit request
-> copied Draft based on prior publication
-> new Published curation
-> previous publication Superseded
```

Only Draft curations can be edited or published. Published curations are not mutated in place. This preserves the accepted state and gives later edits an explicit lineage.

3.5 Publication

Publication establishes the canonical semantic asset used by Loop 2. A published curation can include:

- accepted and rejected artifacts;
- corrected artifact families;
- corrected positions;
- label overrides;
- native-dimension overrides;
- measurement corridors and nodes;
- dimension interval bindings;
- pinch groups and markers;
- publication notes and lineage.

The FloorPlanTemplate stores the active published curation identity. Consumers should use that identity rather than assuming the newest Draft is authoritative.

4. Loop 2 - Adapt

Loop 2 determines whether a published FloorPlan fits a SitePlan and applies the smallest accepted change.

4.1 SitePlan intake

A SitePlan can be imported from DXF or represented by a controlled rectangular simulation. The preview model can contain:

- geometry paths;
- render paths with source layers and entity kinds;
- colors;
- TEXT and MTEXT;
- source unit and conversion factor;
- inferred setback paths;
- buildable-area bounds.

The current buildable-area model is a practical bounds heuristic. It is not a legal parcel engine or a complete topological model of irregular setbacks and easements.

4.2 Structural footprint

Fit analysis uses the dominant wall mass rather than a naive global bounding box. This prevents a small stray wall-layer segment from defining the width or height of the complete house.

The footprint algorithm:

- projects wall-segment mass along each axis;
- removes a small configurable tail;
- snaps the retained extent back to actual wall coordinates;
- returns a stable structural width and height.

This is a bounded heuristic. A genuinely small structural projection can be excluded if it carries less wall mass than the configured threshold, so the result remains subject to operator review.

4.3 Deficit

The fit model separates:

- minimum width reduction;
- minimum height reduction;
- left, right, bottom, and top overflow at the current placement;
- movement-only conditions;
- available capacity by adjustment group.

A plan that fits but is off-center needs movement, not compression. Axis deficit and per-side overflow are therefore treated as related but distinct data.

4.4 Deterministic proposals

The option generator can produce:

- a single-group reduction;
- an even split;
- a capacity-balanced split;
- multiple valid alternatives;
- exact required totals by axis.

Each proposal must satisfy:

- requested axis totals;
- available group capacity;
- supported operation kinds;
- configured numeric tolerance;
- deterministic validation after optional AI ranking.

If there is insufficient authorized capacity, the system blocks the adjustment instead of scaling the complete house or exceeding a marker's limit.

4.5 Accepted adjustment

The accepted placement contains:

- FloorPlan-to-SitePlan scale;
- placement offsets;
- ordered local compression steps;
- marker coordinates and trim amounts;
- recalculated native dimensions;
- input dimensions;
- per-operation impact data.

Application code derives a canonical adjustment recipe from this accepted placement and persists both representations. The recipe becomes the propagation contract for HousePlanSet.

5. HousePlanSet synchronization and export

HousePlanSet groups the canonical FloorPlan with ElectricalPlan, RoofPlan, and Facade/Elevation sheets.

5.1 Ownership rule

The FloorPlan is the canonical geometry owner. Dependent sheets:

- are registered to the FloorPlan;
- consume its accepted placement and adjustment recipe;
- cannot select an independent site-fit answer;
- cannot mutate the FloorPlan automatically.

This prevents different sheets for the same house from drifting toward contradictory dimensions.

5.2 Sheet intake

Dependent-sheet import:

- creates a managed ImportedDocument;
- creates or resolves its MeasurementContext;
- creates a typed PlanSheet;
- associates it with a PlanSetVersion;
- records a sheet lifecycle status.

Classification may use file, name, and layer hints. Operators can correct the type. The source implementation provides a transactional noncanonical-sheet unlink that removes active projections, then registrations, then the sheet and commits once. Treat this capability as partial until executable/runtime validation is complete.

5.3 Registration

Registration maps a dependent sheet into the canonical FloorPlan coordinate system. A SheetRegistration records:

- PlanSetVersion;
- dependent PlanSheet;
- canonical FloorPlanVersion;
- registration method;
- affine or similarity transform;
- confidence;
- warning and rule summary;
- lifecycle status;
- optional whole-plan proof.

Electrical registration uses structural evidence to estimate a similarity transform. A conclusive estimate is still stored as pending confirmation. The Desktop provides a registration review overlay where canonical FloorPlan walls and transformed Electrical walls are displayed in the same coordinate system.

Registration fails closed when:

- required source files cannot be resolved;
- geometry extraction is invalid;
- structural evidence is insufficient;

- multiple non-equivalent transforms remain plausible;
- ownership identities do not match;
- a proof is missing, stale, malformed, or below policy thresholds.

5.4 Whole-plan registration proof

WholePlanRegistrationProof can bind:

- canonical and dependent identities;
- SHA-256 values for both sources;
- horizontal and vertical coverage;
- RMS residual;
- maximum residual;
- proof-policy version and acceptance state.

Free-text diagnostics are not authority. Export code can rely on the proof only when its typed identity, source hashes, policy version, and numeric thresholds are valid.

5.5 Projection

A SheetAdjustmentProjection binds:

- the dependent PlanSheet;
- the confirmed registration;
- the canonical adjustment;
- projection method and transform;
- confidence and status;
- compression-step count;
- rule summary;
- recipe-handling summary.

Projection behavior depends on sheet capability:

- **ElectricalPlan:** supports affine placement and selected local compression operations through registered coordinates.
- **RoofPlan:** supports affine placement; local canonical compression is unsupported.
- **Facade/Elevation:** supports affine placement; local canonical compression is unsupported.

Unsupported semantics remain blocked. Manual confirmation cannot promote an unimplemented transformation into a supported one.

5.6 Export

The multi-sheet export workflow:

- resolves the latest applicable projection for each dependent sheet;
- creates a staging directory;
- copies the canonical FloorPlan output;
- generates supported dependent outputs from their managed sources;
- builds audit data and a typed verification result;
- writes the manifest;
- publishes the final directory atomically;
- persists PlanSetExport and PlanSetExportedSheet snapshots.

Typical package structure:

```
house-plan-set/
house-floorplan.dxf
house-electrical.dxf
manifest.json
audit/
```

The package is not considered automatically ready merely because files exist.

5.7 Fail-closed quality gates

Verification can evaluate:

- expected output presence;
- canonical operation count and impact;
- dependent operation application;
- registration authorization;
- outline evidence;
- segment congruence;
- final-output congruence;
- DXF safety;
- supported sheet capabilities;
- manifest completeness.

A failed or missing gate produces a blocked or manual-review result. The export pipeline does not silently remove the gate to make the package appear successful.

6. Layered architecture

FloorplanFit is a modular local-first monolith with five production projects.

Layer	Responsibility	Must not own
Contracts	Requests, responses, DTOs, geometry projections, audit and verification shapes	Business rules, persistence, UI
Domain	Entities, identities, lifecycle states, invariants, capability and proof policies	SQLite, files, HTTP, Avalonia, IxMilia
Application	Use cases, ports, validation, deterministic rules, transactions, orchestration	UI rendering and infrastructure details
Infrastructure	DXF, SQLite, hashing, filesystem, backups, manifests, registration implementation, AI adapters	Product workflow authority
Desktop	Avalonia views, ViewModels, interaction, navigation, visual review, operator feedback	Direct persistence rules and geometry authority

6.1 Dependency direction

```
Desktop -> Application + Contracts + Infrastructure composition
Infrastructure -> Application ports + Contracts + Domain
Application -> Contracts + Domain
Contracts -> no project dependencies
Domain -> no project dependencies
```

Desktop may compose infrastructure implementations, but use-case decisions belong in Application. Infrastructure may implement a registration estimator, but the acceptance policy and workflow remain controlled by Domain and Application.

7. Data structures and relationships

7.1 Representation boundaries

The system uses three representations:

- **Domain entities and value objects** enforce valid construction and lifecycle state.
- **Contracts DTOs** flatten data for use-case and UI boundaries.
- **SQLite records** preserve durable state and JSON-encoded technical details.

FloorPlanReviewSessionDto, for example, is a Desktop read model that combines multiple persisted families. It is not an aggregate root and does not own their lifecycle.

7.2 Core relationship graph

```
MeasurementContext
<- ImportedDocument
<- FloorPlanVersion <- FloorPlanTemplate
<- WallExtractionRun
<- extracted artifact families
<- FloorPlanCuration
<- classifications and positions
<- label and dimension overrides
<- PinchGroup <- PinchMarker
<- MeasurementCorridor <- MeasurementNode
<- DimensionIntervalBinding

FloorPlanTemplate
-> HousePlanSet
-> PlanSetVersion -> canonical FloorPlanVersion
-> PlanSheet -> ImportedDocument
-> SheetRegistration -> WholePlanRegistrationProof
-> CanonicalFloorPlanAdjustment
-> SheetAdjustmentProjection
-> PlanSetExport -> PlanSetExportedSheet
-> PlanSetAuditEvent
```

7.3 Source and version entities

MeasurementContext

- source unit;
- millimeter conversion factor;
- linear tolerance;
- angular tolerance.

ImportedDocument

- document type;
- original file name;
- managed path;
- SHA-256;
- DXF version;
- MeasurementContext identity.

FloorPlanTemplate

- stable code and name;
- current FloorPlanVersion;

- active published FloorPlanCuration.

FloorPlanVersion

- template identity;
- imported-document identity;
- geometry fingerprint;
- incremental version number;
- soft-delete persistence state.

7.4 Extraction and curation entities

WallExtractionRun owns the status and extractor identity for one FloorPlanVersion.

Artifact entities reference the extraction run and preserve source-facing evidence. Shared `geometry_paths` and `geometry_segments` allow several artifact types to refer to normalized geometry.

FloorPlanCuration owns versioned operator decisions. Its related persistence families include:

- classifications;
- corrected positions;
- label overrides;
- dimension overrides and primitives;
- dimension-binding overrides and anchors;
- pinch groups and markers;
- measurement corridors and nodes;
- dimension interval bindings.

7.5 Fit structures

MeasurementCorridor, also represented in the operator vocabulary as a measurement band or *franja*, defines:

- axis;
- guide path;
- band bounds;
- status;
- ordering.

MeasurementNode anchors a source artifact and geometry path to a corridor. It stores source identity, snap kind, anchor position, axis coordinate, offsets, and normalized position.

DimensionIntervalBinding connects a native dimension to a corridor interval through start and end nodes.

PinchGroup groups authorized compression behavior by axis.

PinchMarker references an eligible source candidate and geometry path, with normalized position and maximum trim.

Measurement structures explain how dimensions relate to geometry. Pinch structures define where deformation is authorized. They are complementary, not interchangeable.

7.6 Site adjustment structures

`SitePlanPreviewDto` carries transient preview geometry, text, unit conversion, and buildable bounds.

`AutoFitEnvelopeDeficitDto` separates axis reduction from current positional overflow.

`AutoFitCandidateGroupDto` exposes authorized capacity.

`AdjustedSitePlanPlacementDto` records accepted affine placement, compression steps, adjusted dimensions, input audit, and operation impact.

`CanonicalFloorPlanAdjustment` durably stores:

- `PlanSetVersion`;
- canonical `FloorPlanVersion`;
- `SitePlan` source path;
- canonical output path;
- placement JSON;
- canonical recipe JSON.

7.7 HousePlanSet entities

HousePlanSet is the stable multi-sheet identity for a `FloorPlanTemplate`.

PlanSetVersion binds one `HousePlanSet` to one canonical `FloorPlanVersion`.

PlanSheet binds a dependent technical sheet to its imported document and measurement context.

SheetRegistration binds the dependent sheet to the canonical coordinate system.

SheetAdjustmentProjection binds registration and canonical adjustment into a sheet-specific result.

PlanSetExport and **PlanSetExportedSheet** preserve export status, output paths, projection lineage, confidence, warnings, and recipe summaries.

PlanSetAuditEvent stores append-oriented JSON events against an aggregate type and identity.

8. SQLite persistence lifecycle

The application uses one local SQLite database with direct SQL, repositories, and Unit of Work.

8.1 Table families

Family	Stored data
Source identity	measurement contexts, imported documents
FloorPlan identity	templates, versions
Extraction	runs, artifact families, paths, segments, dimensions
Curation	curations, classifications, positions, overrides
Fit	pinch groups, markers, corridors, nodes, interval bindings
HousePlanSet	plan sets, versions, sheets
Synchronization	registrations, whole-plan proof JSON, projections, canonical adjustments
Delivery	exports, exported-sheet snapshots, audit events

8.2 Integrity model

The schema combines:

- primary and unique constraints;
- indexes;
- additive schema checks;
- versioned migration;

- targeted identity triggers;
- Application transaction ordering.

Critical PlanSet registration identities are checked against PlanSetVersion and PlanSheet ownership. Canonical FloorPlan versions referenced by PlanSetVersion are protected from deletion.

The schema does not declare a complete foreign-key graph for every relationship. Repositories and Application handlers therefore must preserve operation ordering and commit related changes as one unit.

8.3 Workspace

The local workspace contains:

```
%LocalAppData/
FloorplanFit/
workspace/
app.db
library/
raw-dxf/
backups/

%Desktop%/
exports/ # adjusted-DXF directory supplied by Desktop
```

AppWorkspace can fall back to a workspace-local adjusted-DXF directory when no export directory is supplied:

```
FloorplanFit/
workspace/
library/
adjusted-dxf/
```

Workspace durability includes:

- stable application-data location;
- legacy workspace migration;
- staging;
- reparse-point rejection;
- database backup before migration;
- bounded cleanup within validated parent directories;
- rejection of unsupported future schema versions.

9. Provenance, artifacts, and auditability

Technical lineage is preserved through:

- source path and managed path;
- source hash;
- measurement context;
- FloorPlanTemplate and FloorPlanVersion;
- extraction-run identity;
- curation identity and lineage;
- active publication identity;
- PlanSetVersion and PlanSheet identity;
- registration identity and source proof;
- canonical adjustment identity;
- projection identity;

- export and exported-sheet snapshot;
- manifest and audit files.

Audit artifacts can describe:

- requested input dimensions;
- canonical recipe operations;
- FloorPlan operation impact;
- Electrical registration;
- Electrical projection;
- unsupported or unmatched entities;
- DXF handle and owner safety;
- outline, segment, and final-output evidence;
- quality signals;
- verification reasons;
- human-readable package status.

Observability is not a substitute for authority. An audit is trustworthy only when its evidence selector represents the complete technical subject being evaluated.

10. Desktop and operator workflows

10.1 Library

The Library is the main asset and PlanSet control surface. It allows operators to:

- import FloorPlans;
- inspect templates and versions;
- select a version;
- open review;
- enter Adjust to Site Plan;
- inspect HousePlanSet sheets;
- import a dependent sheet by explicit type;
- correct sheet classification;
- register, confirm, reject, or unlink a dependent sheet;
- review current registration and projection status.

10.2 Review/Edit

The review surface provides:

- CAD-like visualization;
- semantic and layer rendering;
- selection and deletion of false positives;
- restoration of excluded evidence;
- manual corrections;
- dimension editing;
- measurement-corridor controls;

- node and interval binding;
- pinch-group and marker controls;
- compression preview;
- Draft and publication actions.

10.3 Adjust to Site Plan

The adjustment workflow provides:

- import or simulation setup;
- buildable-area inputs;
- SitePlan rendering;
- FloorPlan overlay;
- manual movement;
- fit diagnosis;
- deterministic options;
- optional AI ranking;
- preview of changed dimensions;
- export;
- HousePlanSet package feedback.

10.4 Registration review

Electrical pending confirmation has a dedicated visual review:

- canonical FloorPlan walls;
- transformed Electrical walls;
- shared coordinate system;
- separate colors and opacity;
- registration diagnostics;
- explicit confirm or cancel action.

If the model cannot resolve source geometry or a valid transform, confirmation is disabled.

11. Technology stack and configuration

Concern	Technology
Runtime	.NET, net10.0
Language	C# with nullable reference types
Desktop	Avalonia, Fluent theme
MVVM	CommunityToolkit.Mvvm
Composition	Microsoft.Extensions.Hosting
Persistence	Microsoft.Data.Sqlite
CAD	IxMilia.Dxf behind ports
Tests	xUnit, Microsoft.NET.Test.Sdk, coverlet
Optional AI	OpenAI Responses API through HttpClient

Important configuration:

- OPENAI_API_KEY enables OpenAI assistance;
- FLOORPLANFIT_OPENAI_MODEL overrides the default model;
- the OpenAI base URL is configurable;
- absence or failure of AI does not remove deterministic fit options.

A Claude adapter exists as experimental code and is not part of the default Desktop composition.

NetTopologySuite is not an installed runtime dependency. Geometry behavior currently uses repository-owned segment, extent, and projection logic.

12. Capability matrix

Capability	State	Operating note
Managed FloorPlan import	Implemented	Source copy, hash, units, version identity
Incremental FloorPlan versions	Implemented	Canonical references are protected
CAD artifact extraction	Implemented	Convention-specific rather than universal
Human review and curation	Implemented	Extraction proposes; publication authorizes
Draft/publish/supersede lifecycle	Implemented	Published state is not edited in place
Native dimensions and overrides	Implemented	Source-aware editing and export
Measurement corridors and bindings	Implemented	Axis-aware relationship model
Pinch groups and markers	Implemented	Authorized local compression capacity
SitePlan import and preview	Implemented	Visual content is preserved
Buildable-area interpretation	Partial	Bounds heuristic, not legal topology
Deterministic fit proposals	Implemented	Validated against capacity and totals
AI ranking and explanation	Optional	Never geometry authority
Adjusted FloorPlan/SitePlan export	Implemented	Source-preserving strategy and audit data
HousePlanSet backbone	Implemented	Domain, Application, persistence, Desktop
Electrical registration	Partial	Evidence and confirmation workflow exist; general proof requires continued validation
Electrical recipe projection	Implemented	Supported entity and operation path
Electrical automatic congruence auditor	Partial	Evidence selector can misidentify the complete footprint
Roof affine placement	Implemented	No local compression semantics
Facade/Elevation affine placement	Implemented	No local compression semantics
Roof and Facade compression	Required	Must remain blocked until semantics exist
Automatic stale projection lifecycle	Required	No NeedsReproject transition
Dependent local overrides and replay	Required	No override model
Comparison DXF	Experimental/deferred	Requires a validated source-preserving merger
Continuous integration	Required	Automated baseline remains to be established

13. Failure modes and safeguards

13.1 Source and import failures

Failure modes

- unknown or invalid unit;
- unreadable DXF;
- missing managed copy;
- inconsistent metadata;
- invalid schema ownership.

Safeguards

- explicit MeasurementContext;
- SHA-256;
- managed storage;
- fail-closed constructors and handlers;
- transaction boundaries;
- backup and migration policy.

13.2 Extraction failures

Failure modes

- false wall candidates;
- decorative entities on structural layers;
- missing openings or fixtures;
- duplicated dimensions;
- unsupported entity forms.

Safeguards

- artifact-family separation;
- source references and detection notes;
- operator exclusion and reclassification;
- protected details;
- typed handling for supported DXF entities;
- failure instead of fabricated geometry for invalid supported inputs.

13.3 Fit failures

Failure modes

- stray segment corrupts footprint;
- insufficient compression capacity;
- off-center placement mistaken for deficit;
- option exceeds group limits;
- AI invents an operation.

Safeguards

- structural wall-mass footprint;
- movement and deficit separation;
- deterministic capacity calculation;
- post-AI validation;
- operation totals and tolerance checks;
- block when capacity is insufficient.

13.4 Registration failures

Failure modes

- insufficient whole-plan structural evidence;
- ambiguous transforms;
- local frame mistaken for the house;
- invalid source hash;
- unsupported geometry;
- stale or missing proof.

Safeguards

- connected-frame and whole-plan evidence;
- typed proof policy;
- source identity binding;
- pending human confirmation;
- visual overlay;
- fail-closed export authorization.

13.5 Projection failures

Failure modes

- recipe operation has no dependent geometry;
- operation uses an unsupported entity;
- registration and projection identities do not match;
- sheet lacks compression semantics;
- projection belongs to another canonical adjustment.

Safeguards

- sheet capability policy;
- ownership validation;
- recipe-handling summary;
- operation audit;
- manual-review status;
- no output when required semantics are unsupported.

13.6 Export failures

Failure modes

- invalid handles or owners;
- incomplete dependent output;

- audit mismatch;
- failure after staging;
- partial package publication.

Safeguards

- source-preserving patching;
- DXF safety audit;
- typed verification;
- staging directory;
- atomic publication;
- rollback and staging cleanup;
- persisted failure status.

14. Extension strategy

Extensions should preserve the authority hierarchy and dependency direction.

14.1 Add a new dependent sheet type

Required work:

- define PlanSheetType and capability policy;
- define a registration method;
- implement registration evidence behind an Application port;
- add confirmation workflow;
- define projection semantics;
- define unsupported operations explicitly;
- implement source-preserving export;
- add verification and audit artifacts;
- expose Desktop controls only after the workflow is fail-closed.

14.2 Add a new adjustment operation

Required work:

- define canonical operation semantics;
- bind it to authorized Loop 1 data;
- validate capacity deterministically;
- update FloorPlan preview and export;
- update dimension projection;
- define behavior per dependent sheet type;
- audit affected and unaffected geometry;
- reject sheets that cannot safely replay it.

14.3 Add AI assistance

AI may be added when it:

- ranks deterministic options;

- explains tradeoffs;
- summarizes audit data;
- suggests review targets.

AI must not:

- replace the managed source;
- publish a curation;
- create hidden compression capacity;
- approve registration;
- bypass a verification gate;
- generate an untraceable final DXF.

14.4 Improve SitePlan geometry

The next SitePlan model can add:

- irregular polygons;
- multiple setback families;
- easements;
- polygon validation;
- exact feet-inch parsing;
- distinction between architect-provided and inferred buildable areas.

This should extend the current Contracts and Application boundaries rather than embed zoning logic in Desktop.

14.5 Add reprojection lifecycle

A complete lifecycle should include:

```
ReadyForExport
-> canonical adjustment superseded
-> NeedsReproject
-> reprojection requested
-> projected
-> reviewed
-> ReadyForExport
```

Dependent local overrides should be modeled separately and replayed after reprojection. They must not mutate the canonical FloorPlan.

15. Strategic relationship to Point.ai

Point.ai established an important product lesson: inferred raster geometry is useful evidence, but it is a dangerous final authority when the output contract requires exact, editable, AutoCAD-valid CAD.

Its work exposed the need for:

- human-in-the-loop correction;
- artifact identity and provenance;
- reusable canonical plans;
- site-fit analysis;
- deterministic mutation after approval;
- auditable outputs;

- clear separation between automatic proposal and accepted technical truth.

FloorplanFit preserves those concepts while changing the risk boundary.

```
Point.ai lesson:
raster inference can propose technical evidence,
but global CAD topology and editability require stronger authority.

FloorplanFit response:
real DXF source
-> persisted human-reviewed curation
-> deterministic operations in authorized zones
-> registered dependent sheets
-> auditable export
```

This is not a rejection of AI. It is a rule about where AI belongs.

- AI remains valuable for interpretation, prioritization, ranking, and explanation.
- Human review remains responsible for canonical semantic decisions.
- Deterministic code remains responsible for geometry mutation and validation.
- The managed DXF and its lineage remain responsible for technical provenance.

The result is a system that can adopt better inference later without surrendering its source-of-truth model. A future extractor or model can propose higher-quality artifacts while the curation, publication, fit, projection, and audit boundaries remain stable.

16. Operating principles

- **Preserve the source.** Never destroy the managed DXF to simplify downstream code.
- **Separate evidence from decisions.** Extraction records what was detected; curation records what was accepted.
- **Publish explicitly.** Draft state is not canonical state.
- **Adjust only where authorized.** A fit deficit does not justify global scaling.
- **Propagate one decision.** Dependent sheets consume the canonical recipe.
- **Fail closed.** Missing evidence, unsupported semantics, and ambiguous transforms require review.
- **Make outputs traceable.** Every package should retain identity, recipe, projection, and verification lineage.
- **Keep AI optional.** Loss of AI must not remove deterministic core behavior.
- **Extend by capability.** Add new sheet and operation semantics explicitly rather than weakening shared gates.
- **Prefer operational truth over apparent success.** A blocked package with an accurate reason is safer than a green package produced by the wrong evidence.